\# Plan de Aseguramiento de Calidad de Software (SQA)

\## VotaGT - Sistema de Gestión Electoral

Estándar de referencia: IEEE 730-2014

Curso: Análisis de Sistemas II

Versión: 1.0

Fecha: Octubre 2026

\---

\# 1. Propósito

El presente Plan SQA define las actividades de aseguramiento de calidad aplicadas al desarrollo de VotaGT.

El objetivo es establecer mecanismos para verificar que:

\- Los requerimientos estén definidos.

\- La arquitectura sea consistente.

\- El backend funcione correctamente.

\- Los endpoints cumplan el contrato.

\- Los errores sean controlados.

\- Las pruebas sean reproducibles.

\- Los defectos sean identificados y gestionados.

\- La documentación permanezca alineada con la implementación.

\---

\# 2. Alcance

El plan cubre los siguientes componentes:

\- API REST.

\- Autenticación.

\- Autorización.

\- Votaciones.

\- Candidatos.

\- Mesas.

\- Resultados.

\- Exportación.

\- PostgreSQL.

\- Redis.

\- MongoDB.

\- Neo4j.

\- BDD.

\- OpenAPI.

\- Pruebas automatizadas.

\- Postman.

\- Documentación técnica.

\---

\# 3. Objetivos de calidad

Los principales objetivos de calidad son:

\- Reducir defectos durante el desarrollo.

\- Detectar errores antes de la entrega.

\- Mantener trazabilidad.

\- Validar requisitos funcionales.

\- Verificar requisitos no funcionales.

\- Comprobar seguridad básica.

\- Verificar persistencia.

\- Verificar tolerancia a fallos.

\- Mantener consistencia entre código y documentación.

\---

\# 4. Diferencia entre QA y QC

\## 4.1 Quality Assurance - QA

QA se enfoca en prevenir defectos mediante procesos, estándares y prácticas.

En VotaGT se consideran actividades QA:

\- Definición formal de requisitos.

\- Uso de SRS.

\- Uso de BDD.

\- Revisión de arquitectura.

\- Uso de OpenAPI.

\- Definición de DoR y DoD.

\- Uso de patrones de diseño.

\- Revisión de trazabilidad.

\- Aplicación de estándares.

El objetivo de QA es prevenir problemas antes de que lleguen a producción o a la fase final de pruebas.

\---

\## 4.2 Quality Control - QC

QC se enfoca en detectar defectos mediante ejecución y validación.

En VotaGT se consideran actividades QC:

\- Pruebas unitarias.

\- Pruebas de integración.

\- Pruebas en Postman.

\- Pruebas negativas.

\- Validación de códigos HTTP.

\- Validación de persistencia.

\- Pruebas de caché.

\- Pruebas de Circuit Breaker.

\- Verificación de respuestas RFC 9457.

\---

\# 5. Enfoque Shift-Left

VotaGT aplica un enfoque Shift-Left al introducir actividades de calidad desde las primeras fases del desarrollo.

En lugar de esperar hasta el final para probar el sistema, se realizan validaciones durante:

\- Definición de requisitos.

\- Diseño.

\- Implementación.

\- Integración.

\- Pruebas.

\- Documentación.

Ejemplos aplicados:

\- Requisitos identificados antes de implementar.

\- Escenarios BDD antes o durante el desarrollo.

\- Validación de endpoints mediante Postman.

\- Pruebas automatizadas durante la implementación.

\- Revisión de OpenAPI.

\- Pruebas de patrones arquitectónicos.

\- Revisión de trazabilidad.

\---

\# 6. Estrategia de pruebas

La estrategia combina:

\- Pruebas unitarias.

\- Pruebas de integración.

\- Pruebas funcionales de API.

\- Pruebas negativas.

\- Pruebas de resiliencia.

\- BDD.

\---

\# 7. Pirámide de pruebas

La estrategia sigue una pirámide de pruebas.

```text
&#x20;               /\\
&#x20;              /  \\
&#x20;             / API \\
&#x20;            /-------\\
&#x20;           /Integración\\
&#x20;          /-------------\\
&#x20;         /   Unitarias    \\
&#x20;        /\_\_\_\_\_\_\_\_\_\_\_\_\_\_\_\_\_\_\_\_\_\_\_\_\_\_\_\\
```

\## Nivel 1 - Pruebas unitarias

Son la base de la pirámide.

Validan componentes aislados.

Actualmente se prueban:

\- Permission Middleware.
\- Strategy JSON.
\- Strategy CSV.
\- Circuit Breaker.

Características:

\- Rápidas.

\- Aisladas.

\- Repetibles.

\- Bajo costo de ejecución.

\---

\## Nivel 2 - Pruebas de integración

Comprueban la interacción entre componentes.

Ejemplos:

\- API + autenticación.

\- API + PostgreSQL.

\- JWT + endpoint protegido.

\- Consulta de votaciones.

Actualmente se utilizan pruebas con `node:test`.

\---

\## Nivel 3 - Pruebas funcionales de API

Se realizan mediante Postman.

Validan el sistema desde la perspectiva de un consumidor HTTP.

Ejemplos:

\- Login.

\- Perfil.

\- Votaciones.

\- Candidatos.

\- Mesas.

\- Resultados.

\- Exportación.

\---

\# 8. Pruebas positivas

Las pruebas positivas comprueban caminos exitosos.

Ejemplos:

\- Login correcto devuelve HTTP 200.

\- Crear votación devuelve HTTP 201.

\- Crear candidato devuelve HTTP 201.

\- Registrar mesa devuelve HTTP 201.

\- Registrar resultado devuelve HTTP 201.

\- Exportar CSV devuelve HTTP 200.

\---

\# 9. Pruebas negativas

Las pruebas negativas comprueban condiciones inválidas.

Ejemplos:

\- Login incorrecto.

\- Token inexistente.

\- Permiso insuficiente.

\- Votación inexistente.

\- Número de lista duplicado.

\- Código de mesa duplicado.

\- Resultado duplicado.

\- Votos negativos.

\- Formato de exportación inválido.

\---

\# 10. Pruebas de resiliencia

Se realizan pruebas para verificar tolerancia a fallos.

\## Redis no disponible

Procedimiento:

1\. Mantener API en ejecución.

2\. Detener Redis.

3\. Consultar resultados.

4\. Verificar Circuit Breaker.

5\. Confirmar que PostgreSQL continúa respondiendo.

Resultado esperado:

\- `Redis: DEGRADED`

\- Circuit Breaker pasa a `OPEN`.

\- API continúa respondiendo.

\## Recuperación

Después de restaurar Redis:

\- Circuit Breaker pasa a `HALF\_OPEN`.

\- Si Redis responde, vuelve a `CLOSED`.

\---

\# 11. Pruebas de caché

Se valida Cache-Aside mediante:

```text
CACHE MISS
CACHE GUARDADA
CACHE HIT
```

También se valida la invalidación después de registrar un nuevo resultado.

\---

\# 12. Pruebas de seguridad

Las pruebas básicas de seguridad incluyen:

\- Endpoint protegido sin token -> HTTP 401.

\- Usuario sin permiso -> HTTP 403.

\- Contraseña comparada mediante bcrypt.

\- JWT requerido en recursos protegidos.

\- Credenciales inválidas con respuesta genérica.

\- Variables sensibles fuera del código fuente.

\---

\# 13. Pruebas de errores

Los errores deben utilizar:

`application/problem+json`

y contener:

\- type.

\- title.

\- status.

\- detail.

\- instance.

Se prueban respuestas:

\- 400.

\- 401.

\- 403.

\- 404.

\- 409.

\- 500.

\---

\# 14. Herramientas utilizadas

Las principales herramientas de calidad son:

\- Node.js `node:test`.

\- Postman.

\- Swagger Editor.

\- Gherkin / BDD.

\- PostgreSQL.

\- Redis CLI.

\- MongoDB.

\- Neo4j Browser.

\---

\# 15. Ejecución de pruebas automatizadas

Las pruebas se ejecutan mediante:

```cmd
npm test
```

Criterio de aceptación:

```text
fail 0
```

Una ejecución con pruebas fallidas no deberá considerarse conforme para la entrega final.

\---

\# 16. Evidencias

Las evidencias deberán incluir capturas de:

\- Ejecución de pruebas automatizadas.

\- Postman.

\- Swagger.

\- Redis HIT/MISS.

\- Circuit Breaker.

\- MongoDB.

\- Neo4j.

\- Restauración de PostgreSQL.

Las evidencias pueden almacenarse en:

`docs/evidence/`

\---

\# 17. Trazabilidad de pruebas

Cada prueba deberá poder relacionarse con:

\- Requisito.

\- Historia de Usuario.

\- Endpoint.

\- Escenario BDD.

\- Componente backend.

La matriz correspondiente se encuentra en:

`docs/agile/matriz\_trazabilidad.md`

\---

\# 18. Criterios de entrada

Antes de ejecutar una prueba se deberá verificar:

\- Backend disponible.

\- Dependencias instaladas.

\- Variables de entorno configuradas.

\- PostgreSQL disponible.

\- Datos de prueba disponibles.

\- Token válido cuando corresponda.

Para pruebas de resiliencia Redis podrá detenerse intencionalmente.

\---

\# 19. Criterios de salida

Una funcionalidad podrá considerarse validada cuando:

\- Cumpla sus criterios de aceptación.
\- Responda con el código HTTP esperado.
\- No produzca errores no controlados.
\- Las pruebas automatizadas relacionadas pasen.
\- Exista evidencia cuando corresponda.
\- La trazabilidad esté actualizada.

\---

\# 20. Responsabilidades

\## Analista / Product Owner

\- Validar requisitos.
\- Revisar criterios de aceptación.
\- Mantener backlog y trazabilidad.

\## Desarrollo

\- Implementar funcionalidades.
\- Crear pruebas unitarias.
\- Corregir defectos.
\- Mantener documentación técnica.

\## QA

\- Ejecutar pruebas.

\- Registrar defectos.

\- Validar criterios de aceptación.

\- Comprobar regresiones.

En el contexto académico, una misma persona puede asumir más de uno de estos roles.

\---

\# 21. Gestión de cambios

Cuando se modifique una funcionalidad deberán revisarse:

\- Requisitos.

\- Código.

\- OpenAPI.

\- BDD.

\- Postman.

\- Pruebas automatizadas.

\- Matriz de trazabilidad.

Esto reduce inconsistencias entre los artefactos.

\---

\# 22. Métricas básicas de calidad

Se pueden utilizar las siguientes métricas:

Métrica	Criterio
Pruebas automatizadas fallidas	0
Endpoints críticos probados	100% de los implementados seleccionados
Errores no controlados	0
Requisitos implementados con trazabilidad	100% del alcance implementado
Defectos críticos abiertos al entregar	0

\---

\# 23. Riesgos de calidad

\## Riesgo 1 - Inconsistencia entre código y documentación

Mitigación:

Actualizar OpenAPI, BDD y trazabilidad después de cambios.

\## Riesgo 2 - Fallo de Redis

Mitigación:

Circuit Breaker y fallback PostgreSQL.

\## Riesgo 3 - Datos inválidos

Mitigación:

Validaciones de servicio y restricciones PostgreSQL.

\## Riesgo 4 - Acceso no autorizado

Mitigación:

JWT y middleware de permisos.

\## Riesgo 5 - Regresión funcional

Mitigación:

Pruebas automatizadas y Postman.

\---

\# 24. Conclusión

El Plan SQA de VotaGT integra actividades preventivas y de control durante todo el ciclo de desarrollo.

El enfoque Shift-Left permite detectar problemas tempranamente, mientras que la combinación de pruebas unitarias, integración, API, BDD y resiliencia proporciona evidencia verificable del comportamiento del sistema.

La calidad no se limita a la fase final, sino que se incorpora desde requisitos, diseño e implementación hasta pruebas y documentación.